

The `lutorch_ex` library

lookup-table layers: tables, projections, cartridges, and results

October 2026

In one paragraph. `lutorch_ex` is a small PyTorch library of *lookup-table layers*: drop-in replacements for a feed-forward layer that, instead of multiplying the input by weight matrices, turn it into a few sign bits, use those bits as addresses into many small learnable tables, and add up the rows they read. This report walks through it in four parts: how a table is addressed and read, and how tables form a layer (§1); a projection wrapper that gives a stack of tables an ordinary vector-in, vector-out interface, so it can sit anywhere a feed-forward layer would (§2); *cartridges* — interchangeable read mathematics behind one fixed contract, in three generations, one of which is quantised to int8 and ships with a deployment path (§3); and the two regularisers every cartridge shares (§4). §5 reports a seven-configuration sweep at 48k steps.

1 The idea: tables addressed by an LSH

A lookup table (LUT) replaces a multiply-accumulate with an *address* and a *read*. Hash the input to a small integer with a locality-sensitive hash (LSH) — here, a handful of sign bits — and fetch the stored vector at that address. The construction comes from Izhikevich’s *Spiking Manifesto* [1]: a spiking network stores a vast number of synapses but touches only those of the neurons that fired, which is exactly what a table read does, and the *order* of spikes rather than their magnitude carries the information, which is exactly what a sign-bit address encodes.

One table. Work in a space $z \in \mathbb{R}^{d_{\text{in}}}$ (Figure 1). An *anchor* is an index into z . A table owns `nap`¹ *probes*; probe j is either an anchor pair (a_j, b_j) , giving $u_j = z[a_j] - z[b_j]$, or a single anchor a_j , giving $u_j = z[a_j]$. Each probe contributes one bit $[u_j > 0]$, so an address $c \in [0, K)$ has `nap` bits and the table has $K = 2^{\text{nap}}$ cells, each holding a learnable row $W[c] \in \mathbb{R}^{d_{\text{out}}}$. The magnitudes $|u_j|$ are the *margins*: how far each bit is from flipping. The smallest one, $u^* = \min_j |u_j|$, marks the least certain bit, and flipping that bit gives the *neighbour* cell c' — the one the input would land in under the smallest perturbation. Every read mathematics in this library — every *cartridge*, in the vocabulary of §3 — uses c , u , and c' and nothing else from the input.

Many tables, many heads. Tables are grouped into G *heads* of `tph`² tables each, and the heads together make up a *layer*, the unit that replaces a feed-forward layer. A head reads its own d_{in} -wide slice of the input, and its output is the *sum* of its `tph` table reads (weighted, in some cartridges). The G head outputs are then *concatenated* into the layer’s output, $G d_{\text{out}}$ wide. A layer may instead fan all heads into one shared output by summing them.

¹`nap`: the number of anchor probes per table.

²`tph`: tables per head.

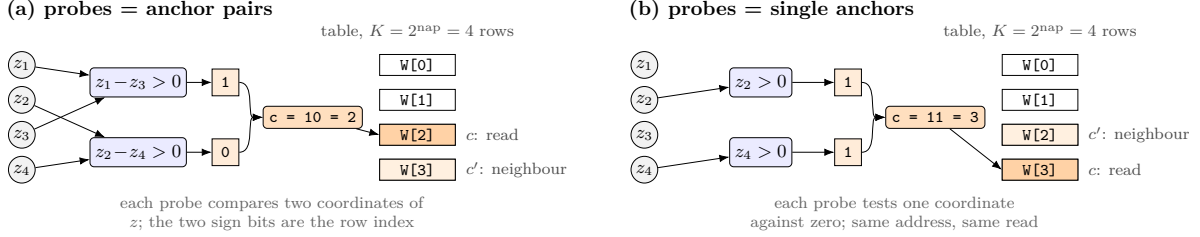


Figure 1: One lookup table in each of the two modes, $\text{nap}=2$. A probe yields one sign bit; the bits, most significant first, form the row index c , which selects one learnable row of the $K = 2^{\text{nap}}$ -row table. The probe with the smallest margin is the least certain, and flipping its bit names the neighbour row c' (lighter), which the richer read forms described later also fetch. (a) Anchor pairs compare two coordinates of z ; (b) single anchors test one coordinate against zero. A head sums many such tables.

Two kinds of probe. Which kind a layer uses is fixed when the layer is built. In the default, pair form, every probe compares two coordinates of z : this is the manifesto’s Eq. 1, where the bit records which of two neurons fired first. In the single form every probe tests the sign of one coordinate instead.

2 ProjectionLUT: input and output projections

ProjectionMHL (the “ProjectionLUT” wrapper, Figure 2) puts a plain linear map on each side of the cartridge — the stack of tables together with its read mathematics — so that, from the outside, it takes an ordinary vector of width D_{in} and returns one of width D_{out} , like any feed-forward layer, while the tables work in their narrow per-head spaces. The two widths are independent arguments of the wrapper and may differ. Writing h_{in} and h_{out} for the number of input and output heads and d_{in} , d_{out} for their widths ($h_{\text{in}} = h_{\text{out}} = G$ in the usual per-head case³):

$$x \in [B, D_{\text{in}}] \xrightarrow{\text{compress}} z \in [B, h_{\text{in}}, d_{\text{in}}] \xrightarrow{\text{cartridge}} y \in [B, h_{\text{out}}, d_{\text{out}}] \xrightarrow{\text{decompress}} \text{out} \in [B, D_{\text{out}}].$$

- **The input projection shapes the hashing.** A probe is also a hyperplane test, and what plane it tests depends on whether **compress** is in front and on the kind of probe:

	single anchors	anchor pairs
compress off	$x[a] > 0$: an axis-aligned plane of the raw input	$x[a] - x[b] > 0$: coefficients ± 1 , zero bias — the manifesto’s form
compress on	$w_a^\top x + b_a > 0$: an arbitrary learned hyperplane	$(w_a - w_b)^\top x + (b_a - b_b) > 0$: also arbitrary; up to $\binom{d_{\text{in}}}{2}$ distinct planes per head

With **compress** on, a single-anchor probe reads one row of the projection and is exactly the manifesto’s “hyperplane LSH” with learnable anchor vectors (1, §VIII-H). With **compress** in front, both modes get arbitrary hyperplanes, but the same projection yields far more distinct planes in pair form, so tables stop re-using the same few bits; single anchors are somewhat weaker, though not dramatically. Without **compress** the gap is large: a single anchor only tests the sign of one raw coordinate, an axis-aligned split, where a pair still compares two.

³The two head counts may differ only when one of them is 1; the library rejects every other combination. With $h_{\text{in}} = 1$ the input projection produces one d_{in} -wide slice that all G heads read (fan-out); with $h_{\text{out}} = 1$ the heads’ outputs are summed into one d_{out} -wide slice that the output projection consumes (fan-in).

- **The output projection compresses the tables.** A layer holds an exponential number of cells — `tph` tables per head, $K = 2^{\text{nap}}$ cells each, times the heads, so the count is exponential in `nap` — and every cell stores a d_{out} -wide row, so the per-cell width d_{out} is what controls table size. For example, with 8 heads of 64 tables and `nap`= 8, that is $8 \times 64 \times 256 = 131,072$ cells per layer, 6.3M table parameters at $d_{\text{out}} = 48$; in a six-block model of width 384 that is 37.7M over six layers, more than half of its 68.2M parameters, and at the full model width, $d_{\text{out}} = 384$, it would be 50M per layer. So the tables live in a narrow d_{out} , and **decompress** expands the concatenated head outputs back to model width afterwards, at the cost of one small dense map per layer (0.15M parameters).

Given a learned input projection, the single-anchor mode alone would have sufficed in principle: a pair comparison $z[a] - z[b] > 0$ is itself a hyperplane on x , so in principle anchor pairs could have been dropped and the pair form left to **compress** to realise. In practice it would not be: ordinary training has no reason to drive the **compress** matrix into a $\{+1, -1\}$ pair form, so recovering it would take machinery this library does not have, such as strict regularisation or a straight-through estimator onto ± 1 weights.

The library keeps both anchor modes deliberately. The pair comparison is the Spiking Manifesto’s mechanism as stated — the relative order of two coordinates — and it is the cheap special case that the whole design is built to exploit: an address bit costs two coordinate reads and one subtraction, with the anchors stored as integer indices and no weights of its own (the kernels do exactly this), whereas a general per-table hyperplane would cost a d_{in} -length dot product and d_{in} weights per bit — in the example above, with $d_{\text{in}} = 48$, some 4,000 subtractions per token and layer against about 200,000 multiply-adds. Single anchors were added later for flexibility, as the even cheaper probe, and sit at the other end of the same trade.

Either projection — the input projection **compress** or the output projection **decompress**, but not both — may be switched off (Identity) when the widths already match. By default both are present, **decompress** is initialised to zero and **compress** to $\mathcal{N}(0, 0.02)$.

In code, the wrapper takes any cartridge — the swappable read mathematics of §3, here **ConfidenceLUT** — and maps $[N, D_{\text{in}}]$ to $[N, D_{\text{out}}]$, so a sequence is flattened first:

```
import torch
from spiky.lutorch_ex import LUTSpec, ConfidenceLUT, ProjectionMHL

spec = LUTSpec(h_in=8, h_out=8, tph=64, nap=8, d_in=48, d_out=48) # 8 heads of 64 tables
cart = ConfidenceLUT(spec, seed=1, read_top_n=2) # the read mathematics
ffn = ProjectionMHL(cart, d_model=384) # or input_dim=..., output_dim=... (may differ)
x = torch.randn(4, 128, 384) # (B, T, D) activations
y = ffn(x.reshape(-1, 384)).reshape(x.shape) # torch.Size([4, 128, 384])
```

Any other cartridge of §3 can take **ConfidenceLUT**’s place.

3 Cartridges

Everything described so far is the same for every read form: the geometry, the addressing, the sum over tables, the routing, the wrapper. A *cartridge* is the swappable read *mathematics* inside that frame. The contract is a shape, $[B, h_{\text{in}}, d_{\text{in}}] \rightarrow [B, h_{\text{out}}, d_{\text{out}}]$ (**MultiHeadLUT**), and the wrapper never reaches inside it. All cartridges are built on one base class, **ManifestoLUT**. It holds the tables $W \in \mathbb{R}^{G \times \text{tph} \times K \times d_{\text{out}}}$ and the anchors (drawn once from a seed, never trained), and it provides what every cartridge needs: the addressed cell, the margins, the runner-up, and at the end the

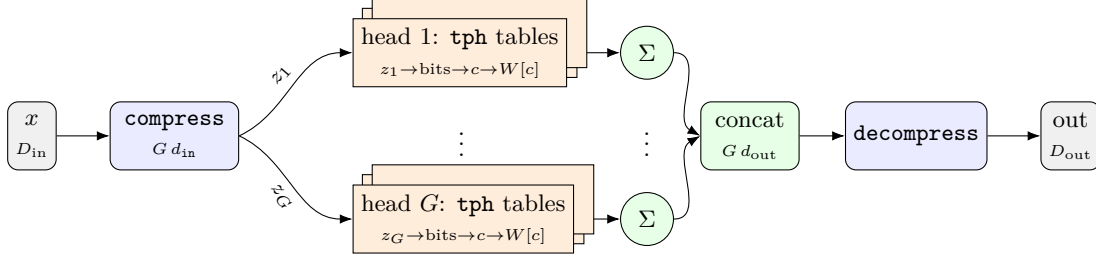


Figure 2: The ProjectionLUT data path. The input projection **compress** ($D_{\text{in}} \rightarrow h_{\text{in}} d_{\text{in}}$) splits x into per-head spaces z_g ; in each head the probes hash z_g to a cell per table and that head’s **tph** reads are summed (Σ); the G head outputs are concatenated, and the output projection **decompress** ($h_{\text{out}} d_{\text{out}} \rightarrow D_{\text{out}}$) maps the concatenation to the output. (With fan-in, $h_{\text{out}} = 1$, the heads are summed into one slice instead.) D_{in} and D_{out} are independent and may differ. The tables and per-head sums are the cartridge; the wrapper treats it as opaque.

routing of each head’s result to its output slot. A cartridge decides the rest: how the addressed row and its neighbour are weighted and added up into a head’s output, and which function training differentiates. Gen-1 and Gen-2 also come as **Fused...** twins with identical mathematics and faster kernels.

3.1 One read, three ways to weight it

Every cartridge computes, per head,

$$y = \sum_t (a_t W_t[c_t] + b_t W_t[c'_t]),$$

with c the addressed cell, c' its least-certain-bit neighbour, and weights a, b that depend on the margins only. The generations differ in a and b (Table 1). The addresses are integers and carry no gradient, so the input gradient flows through a and b alone.

Hard and soft. A *soft* read uses the formula above as its value at train and eval, so both rows learn. A *hard* read returns $W[c]$ alone ($a = 1, b = 0$), adds rows without a multiply, and uses the soft formula only as a *surrogate* for the input gradient; the table update stays on row c . Gen-1 and Gen-2 come in both forms. Gen-3 has no hard form: its weighted read is always the value.

generation	a, b	learned scalars	cartridges (hard / soft)
1 Manifesto (§3.2)	$1 - U, U$	—	ManifestoHardLUT / ManifestoSoftLUT
2 SoftSign (§3.3)	$1 - w, w$	$T_{\text{soft}}, T_{\text{sel}}$	SoftSignHardLUT / SoftSignSmoothLUT
3 Confidence (§3.4)	$s(1 - v), s v$ ($n=2$); $s, 0$ ($n=1$)	β, γ ($+\tau$)	no hard form: ConfidenceLUT, int8 QuantisedConfidenceLUT (§3.5)

Table 1: The three generations as weights on the addressed row and its neighbour. Gen-1 and Gen-2 each also have **Fused...** twins of both forms.

3.2 Gen-1 Manifesto

The Spiking Manifesto’s model [1], with the paper’s fixed uncertainty function of the deciding margin $|u_{j^*}| = u^*$:

$$U = \frac{0.5}{1 + u^*} \in (0, \tfrac{1}{2}], \quad \frac{\partial \tilde{y}}{\partial z[a_{j^*}]} = -\frac{\partial \tilde{y}}{\partial z[b_{j^*}]} = U'(u^*) \operatorname{sgn}(u_{j^*}) (W[c'] - W[c]),$$

so the input gradient lands on the two anchors of the deciding probe alone (manifesto Eqs. 4–5). **ManifestoHardLUT** is Izhikevich’s rule verbatim: its value has $a = 1$, $b = 0$, and $a = 1 - U$, $b = U$ serve only as the surrogate. **ManifestoSoftLUT** promotes the surrogate Eq. 4 to the value, $a = 1 - U$, $b = U$, which goes beyond the paper: there U only computes training gradients, and inference reads with Eq. 3 alone (manifesto §III-A).

3.3 Gen-2 SoftSign

The fixed U becomes a blend weight with two temperatures learned per layer (init 0.5, freezable):

$$\rho = \frac{u^*}{T_{\text{soft}} + u^*} \in [0, 1), \quad w = \sigma(-2\rho/T_{\text{sel}}) \in (0, \tfrac{1}{2}].$$

SoftSignSmoothLUT reads with $a = 1 - w$, $b = w$; **SoftSignHardLUT** reads $a = 1$, $b = 0$ and uses $a = 1 - w$, $b = w$ as its surrogate.

3.4 Gen-3 Confidence

Gen-3 weights each table by a *confidence score* s , the score of LookupFFN [2], computed from all **nap** margins $m_j = |u_j|$ of the table’s address:

$$s = \left(\sum_j m_j \right) \exp\left(\gamma \sum_j \log \sigma(\beta m_j) \right), \quad v = \sigma(-2u^*/\tau) \in (0, \tfrac{1}{2}].$$

The sum of the margins grows with how far the input sits from the bit boundaries; the second factor is the product of $\sigma(\beta m_j)^\gamma$ over the margins, near 1 for a margin far from its boundary and $2^{-\gamma}$ for a margin of zero, so every uncertain bit discounts the table. β sets what counts as a large margin, γ how strong the discount is. β , γ (and τ) are learned scalars, one each per layer, shared by all its tables and stored in log form (init 2, 1, 0.5). With $n=1$ the scored cell is the whole read, $a = s$, $b = 0$, no blend. With $n=2$ the score multiplies a two-cell blend by uncertainty, as in Gen-2: $a = s(1 - v)$, $b = s v$, where the neighbour weight v depends only on the deciding margin u^* and τ . The key property: the input gradient reaches z *only* through s (all **nap** margins) and, for $n=2$, through v (the deciding margin). A table learns how confident to be, never that a different row would have been better.

3.5 The int8 read and deployment

Why. The tables are most of a LUT model: in the example of §2, 6.3M table parameters per layer, 37.7M of the model’s 68.2M. Stored as int8 instead of fp32 they take a quarter of the memory. **QuantisedConfidenceLUT** (quant_mode="p2_int8") is the Gen-3 cartridge ($n=2$) trained for that deployed form.

What p2_int8 is. Two roundings turn the read $s(1 - v) W[c] + s v W[c']$ into integer work:

1. **int8 cells.** Each row is stored as $\hat{W} = \text{clamp}(\text{round}(W/2^e), -128, 127)$, with one power-of-two scale per group and output channel, $e = \lceil \log_2 \max |W| \rceil - 7$ (the maximum over all of the group’s tables and cells), so the largest entry just fits in eight bits.
2. **Power-of-two weights.** Write the read as $s(1 - v) (W[c] + r W[c'])$ with $r = v/(1 - v) = e^{-2u^*/\tau}$. Round r to 2^{-q} , $q = \text{round}(2u^*/(\tau \ln 2))$, and $s(1 - v) = s/(1 + r)$ to $2^{k'}$, $k' = \text{round}(\log_2 s - c_q)$ with $c_q = \log_2(1 + 2^{-q})$. A table then adds $2^{k'} \hat{W}[c] + 2^{k'-q} \hat{W}[c']$: two shifts and an add, no multiply.

3. **Integer sum.** Shifts are counted in units of 2^{-6} , so all are left shifts into an int32 accumulator; the only float left is the per-channel 2^{e-6} , applied once after the table sum.

The exponents q and k' are computed from the margins in floating point; the table read itself is integer. q is capped at 64 and c_q taken as 0 for $q \geq 8$; k' is kept in $[-3, 4]$ (below, the table is skipped but keeps its gradient; above, k' is clamped); the neighbour cell is dropped when $q > 3$.

Training is quant-aware. The training forward already reads the rounded tables with the rounded weights. Both roundings are straight-through: the value is the rounded read, while the gradient is that of the float master tables and of the exact weights $s(1-v)$, sv , so β, γ, τ keep learning.

Deployment. Because training is quant-aware, the deployed numbers are the trained numbers, up to round-off: every training forward already computes with the int8 tables $\hat{W}$ and scales 2^e derived from the float masters. Export is a change of storage, not of the model: `export_deployment` stores those int8 values, drops the float masters, and bakes the scale 2^{e-6} *statically* into a copy of the layer’s `decompress` weight — exact, since a power of two changes only a float’s exponent; no runtime hook applies a scale afterwards. The file holds the int8 tables, the anchors and the three scalars: 24 MiB $\rightarrow$ 6 MiB per layer in the example. `load_deployment` builds the model on PyTorch’s `meta` device and fills it from the file, so the float tables are never allocated. The deployed layer returns the integer accumulator, and `decompress` applies the folded scale.

```
export_deployment(model, "m.lxq")
dep = load_deployment("m.lxq", make_skeleton, device="cuda").eval()
```

3.6 Which to use

For accuracy, `ConfidenceLUT` $n=2$. For deployment, its int8 twin `QuantisedConfidenceLUT`. Gen-1 and Gen-2 are reference points; a hard read is the cheapest possible inference, at a cost in accuracy against its soft twin.

4 Regularisers

Two regularisers live in the shared base class `ManifestoLUT`, so every cartridge has them; both are off by default.

Cell total variation. The $K = 2^{\text{nap}}$ cells of a table are the corners of an `nap`-dimensional cube: two cells are neighbours when their addresses differ in one bit, as c and c' do. The penalty of a layer is the mean squared difference between neighbouring rows,

$$\mathcal{R}_{\text{TV}} = \frac{1}{T \cdot \text{nap} \cdot 2^{\text{nap}-1}} \sum_{t=1}^T \sum_{(c,c') \text{ Hamming-1}} \|W_t[c] - W_t[c']\|_2^2,$$

over all $T = G \cdot \text{tph}$ tables and every neighbouring pair, each counted once (`ManifestoLUT.cell_tv`). `cell_tv_penalty(model)` averages it over the model’s LUT layers, and the training loss adds λ times that. It acts on the table weights only and leaves the forward pass unchanged. The prior it encodes: a one-bit change of address, the smallest change of input, should change the read only a little.

Table dropout. With `table_dropout_rate` $p > 0$, training drops whole tables: one Bernoulli draw per sample, head and table keeps each table with probability $1 - p$, and the survivors are scaled by $1/(1 - p)$, so the expected read is unchanged. A dropped table contributes nothing to that sample’s value or gradient; in Gen-3 the mask multiplies the score s , to the same effect. At evaluation, or whenever gradients are off, nothing is dropped and nothing is rescaled.

5 Experiments: the seven-configuration sweep

Setup. A six-block GPT ($d_{\text{model}} = 384$, 6 attention heads, 68.2M parameters) with the feed-forward layer of every block replaced by a **ProjectionMHL** around the cartridge under test. Every configuration here is square: the wrapper’s input and output widths both equal the model width, $D_{\text{in}} = D_{\text{out}} = d_{\text{model}} = 384$. All seven runs⁴ share the same topology — $h_{\text{in}} = h_{\text{out}} = 8$, `tph`= 64, `nap`= 8, $d_{\text{in}} = d_{\text{out}} = 48$, anchor pairs — seed 1, the default init (per-head Uniform $[\pm 10^{-3}]$ tables, decompress zeroed), cell TV $\lambda = 10$ and table dropout $p = 0.2$ (§4), AdamW (3×10^{-4} , weight decay 0.1, none on the tables), 48,000 steps, cosine schedule with 10% warm-up to a $0.1 \times$ floor, sequence length 512, 24,576 tokens per step (device batch 12, accumulation 4), evaluation every 500 steps; the backbone runs in bf16, the LUT layers in fp32. Data and tokenisation are the *NanoChat* setup: the ClimbMix text corpus as prepared by nanochat’s data script, streamed and tokenised on the fly by nanochat’s RustBPE tokenizer (32,768-entry vocabulary, trained by nanochat on the same corpus) into BOS-aligned 512-token rows; `val_bpb` is the validation cross-entropy in bits divided by the number of *bytes* of the decoded text (nanochat’s bits-per-byte evaluation on a fixed held-out set), so it is comparable across tokenisers.

Reading the results. Five things are visible in Table 2 and Figure 3; each is one run per configuration. (i) In the two generations that come in a hard and a soft form, the *soft* read is better: by 0.012 bpb for Manifesto and 0.027 for SoftSign. (ii) Within Gen-3, also reading the neighbour cell ($n=2$) beats the single scored cell ($n=1$) by 0.011. Both are soft, scored reads, so this compares one cell with two, not hard with soft. (iii) Adding the *confidence score* beats a blend by uncertainty alone: Confidence $n=2$ leads the best SoftSign run by 0.023 and the best Manifesto run by 0.033. The order of all seven runs is fixed from step 7,500 on. (iv) *Quantisation is nearly free*: int8 rows and power-of-two weights cost +0.0012 bpb, for tables one quarter the size. (v) Against the dense feed-forward layer, the two scored two-cell cartridges *win* (−0.009 and −0.008 bpb), the single-cell scored read is within 0.002, and the blend-only generations trail by 0.014–0.041; the LUT models carry $1.9 \times$ the parameters of the dense one, but a LUT layer touches only two 48-wide rows per table per token.

5.1 Benchmarks: CPU, RTX 5090, H100

Table 3 times the layer alone, at the sweep geometry, for the seven optimised configurations the sweep used: the four fused twins and the three Gen-3 configurations (which have no separate twin: their `embedding_bag` read with a compiled forward is already the fast path). Gen-1 and Gen-2 also exist as plain-PyTorch *reference* cartridges (`ManifestoHardLUT` and so on): written for readability, and the oracle the fused twins are tested against for equal results. They are not tuned for speed

⁴Configs, full metric histories and summaries (including wall-clock hours and peak memory) of every run are on the branch, under https://github.com/anatoli-starostin/spiky/tree/feature/lutorch_ex_report/experiments/lutorch_ex.

feed-forward layer	read	params	best val_bpb	Δ vs. dense
dense FFN, $384 \rightarrow 1536 \rightarrow 384$, GELU	—	35.8M	1.11509	—
ConfidenceLUT $n=2$	scored two-cell blend, float	68.2M	1.10606	-0.0090
QuantisedConfidenceLUT $n=2$	the same, int8 + power-of-two	68.2M	1.10724	-0.0079
ConfidenceLUT $n=1$	scored single cell	68.2M	1.11737	+0.0023
FusedSoftSignSmoothLUT	learned-temperature blend	68.2M	1.12941	+0.0143
FusedManifestoSoftLUT	U -blend	68.2M	1.13949	+0.0244
FusedManifestoHardLUT	hard read	68.2M	1.15152	+0.0364
FusedSoftSignHardLUT	hard read	68.2M	1.15647	+0.0414

Table 2: The 48k sweep, one seed, against the dense feed-forward baseline: the same backbone (6 blocks, $d_{\text{model}} = 384$, 6 attention heads), data, tokenizer, schedule, 24,576-token batches and seed, with the dense $384 \rightarrow 1536 \rightarrow 384$ GELU layer in the feed-forward slot (run `exp_n_0177_vanilla48k_corrected` on branch `research/ffn_replacement_fix`).

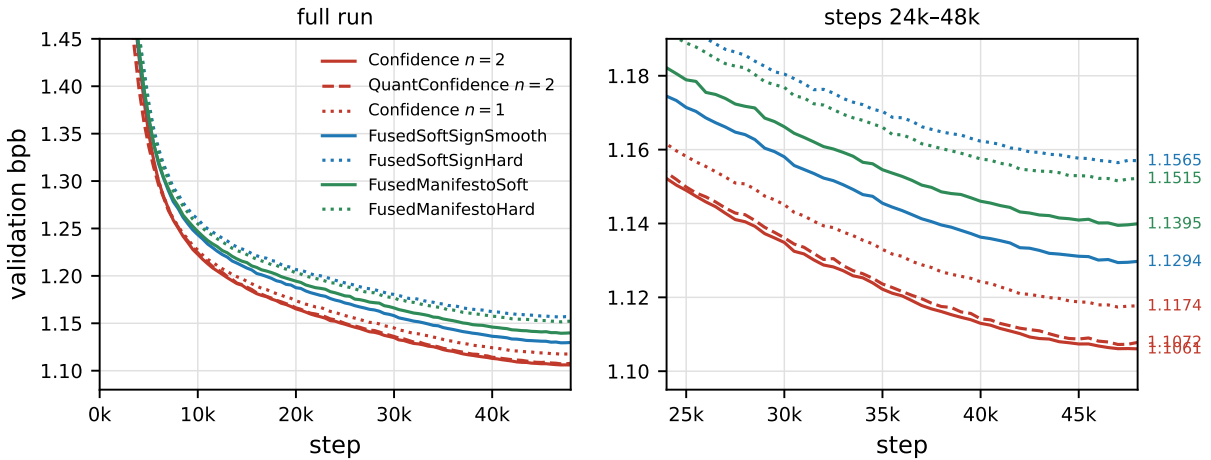


Figure 3: Validation bpb of the seven runs (left: full run; right: the last half, with each run’s best value). Colour is the generation (red Confidence, blue SoftSign, green Manifesto); dotted lines are the one-cell reads (the two hard reads and Confidence $n=1$), dashed is the int8 cartridge.

and the sweep did not train with them, so the table leaves them out. The CPU and RTX 5090 columns were measured for this report with the library’s own harness (`bench.py`, driven by `bench_report.py`, which lives with the run results on the branch named in the footnote of §5, under `doc/research/lutorch_ex_report/`).

Three readings. On either GPU the training step of every blend or hard cartridge is 20–29 ms at the training batch and is dominated by the backward (17–19 ms), the scatter into the 6.3M-entry table; **ConfidenceLUT** $n=1$ is both the cheapest step (11.5 ms) and the smallest footprint (1.1 GB), because its compiled forward fuses the score’s chain of small kernels and reads one cell. Reading two cells ($n=2$, the soft forms) roughly doubles the eval forward (1.0 $\rightarrow$ 3.5 ms for Manifesto, 1.5 $\rightarrow$ 4.6 ms for Confidence). The quantised cartridge’s eval forward is not faster than the float one — its training-time read still fetches float rows for the straight-through estimator; the integer read belongs to the deployed cartridge (§3.5). The fused twins earn their place: **FusedSoftSignHardLUT**’s native training step is $1.6\times$ faster and uses $2.7\times$ less memory than its reference cartridge **SoftSignHardLUT** at the training batch (recorded when the twin landed; the reference cartridges are not in Table 3). The two GPUs agree to within 15% on every row: these layers are bound by memory traffic, not arithmetic. On the CPU, at a batch $24\times$ smaller, the same layers take 36–92 ms per step — roughly

cartridge	CPU (measured)	RTX 5090 (measured)			H100 (recorded, PR #138)		
	step ms, $B=1024$	eval ms	step ms	peak GB	eval ms	step ms	peak GB
FusedManifestoHardLUT	62.9	1.04	20.2	3.8	1.61	21.3	4.0
FusedManifestoSoftLUT	92.3	3.50	22.5	2.3	3.84	23.6	2.5
FusedSoftSignHardLUT	68.8	1.03	24.6	4.2	1.61	23.8	4.2
FusedSoftSignSmoothLUT	60.8	2.13	28.8	2.9	4.10	28.4	3.0
ConfidenceLUT $n=1$	35.8	1.51	11.5	1.1	2.40	12.9	1.2
ConfidenceLUT $n=2$	66.7	4.59	21.5	1.9	4.17	24.7	2.1
QuantisedConfidenceLUT $n=2$	83.4	4.12	23.9	1.8	4.23	24.5	2.0

Table 3: Layer-only timings at $h_{\text{in}} = h_{\text{out}} = 8$, $\text{tph} = 64$, $\text{nap} = 8$, $d_{\text{in}} = d_{\text{out}} = 48$, fp32, random input. *eval* = no-grad forward; *step* = training forward + backward. GPU columns at $B = 24,576$ (the sweep’s training batch). **Measured here:** one workstation, torch 2.9.1 (cu130), CUDA 13.3 toolkit for the JIT kernels, driver 595; RTX 5090 (32 GB, native kernels active): medians of 10 repeats after 3 warm-ups; CPU (24-thread desktop part, the fused twins’ `embedding_bag` path, $B = 1,024$): medians of 15 repeats after 5 warm-ups, with the fastest repeat within 15% of the median (`bench_cpu.json` and `bench_cuda.json` beside that script hold every cell). H100 column measured on an H100 VM provided by Nebius.

two orders of magnitude slower per row — with the same ordering: the one-cell scored read is the cheapest, the two-cell blends cost $1.7\text{--}2.6\times$ more.

References

- [1] E. M. Izhikevich. “Spiking Manifesto.” arXiv:2512.11843, v2, December 2025.
- [2] Z. Zeng, M. Davies, P. Pulijala, K. Sankaralingam, V. Singh. “LookupFFN: Making Transformers Compute-lite for CPU Inference.” ICML 2024.